

Event-Driven Programming



Mr. E. O. Tugbeh

AI Product Engineer, HWP Labs.

About Me

- ❖ Bachelor's degree in Mathematics & Computer Science
- ❖ Master's degree in Computer Science (Advanced Software Engineering)
- ❖ Over 12+ years of experience in **Enterprise Software Development & Training**
- ❖ Worked across critical industries such as Banking, Education, Government & Healthcare
- ❖ Member of Computer Professionals (Registration Council of Nigeria) [CPN]
- ❖ Member of Google Developer Group (GDG) & Student Ambassador 2013/2014
- ❖ Microsoft Certified Azure Developer Associate (AZ-204)
- ❖ Claude Certified Solutions Architect & Discord Server Community Mentor



Overview

0

**Role of
JavaScript**

1

**JavaScript
Fundamentals**

2

**BOM BOM
& DOM**

3

**Events Triggers
& Handlers**

4

**Vibe Coding
w/Claude AI**

Prerequisites

0

**HTML/CSS
Fundamentals**

1

**Charged
Laptop**

2

**Internet
Connection**



A large, white, serif capital letter 'O' is positioned on the left side of the slide. It is set against a background of orange dots of varying sizes.

Role of JavaScript

Webpage Interactivity

“

JavaScript is an **interpreted**,
high-level programming language
used to develop interactive webpages.

Trivia

- ❑ **1995** - JavaScript was created by Brendan Eich (Netscape, *within 10 days*).
- ❑ **2009** - Node.js was created by Ryan Dahl as a JS runtime environment to execute JS outside web browsers.
- ❑ **2011** - React.js was created by Jordan Walke (Facebook) as an open-source JS library for building single-page applications (SPAs).

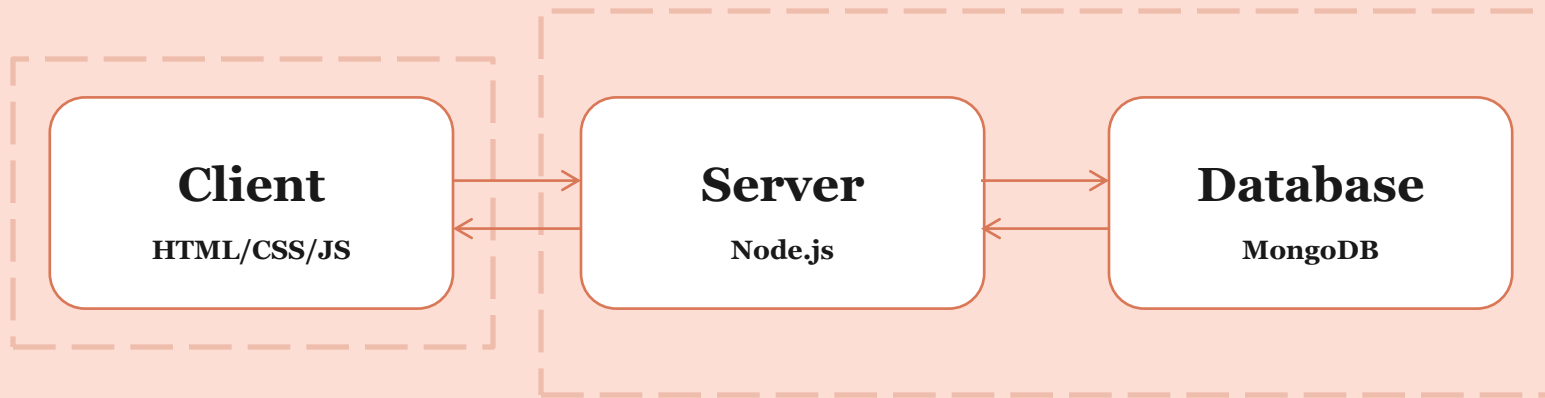


Application Areas

- ❑ Manipulate HTML & CSS (DOM Elements)
- ❑ Handle Webpage Interactions
- ❑ Develop APIs/Web Services (Node.js)
- ❑ Build Desktop Apps (Electron)
- ❑ Build Mobile Apps (React Native)
- ❑ Interface with ETH Smart Contracts (Web3.js)



Web App Architecture





Questions

1

JavaScript Fundamentals

Syntax & Semantics

- ❑ **Variables & Constants** – Named storage location in memory that can/cannot be changed after declaration (var, const).
- ❑ **Data Types** – Defines the type of value to be stored in memory, how much memory it consumes, and what type of operations can be performed on it;
 - ❑ Primitives (Number, String, Boolean, null, undefined)
 - ❑ Non-Primitives (Array, Object, Date)
- ❑ **Operators** – Symbols used to perform computations;
 - ❑ Arithmetic & Assignment
 - ❑ Comparison, Logical & *Ternary

Variables & Constants

1.1

Example 1

1. `// TODO: Compute Student Grade`
2. `var score = 47;`
3. `const total = 60;`
4. `const grade = (score * 100) / total; // 78.33%`
5. `const passed = score >= (total / 2); // true`
6. `const remarks = passed ? "Pass" : "Fail";`
7. `console.log(grade, passed, remarks);`

Identifiers

1. The name given to a variable is known as an **Identifier**.
2. Identifiers can begin with or contain special characters like \$ and _.
3. Numbers cannot be used as the first character of an Identifier.
4. The JavaScript convention for Identifiers is ***camelCase**.
5. Identifiers should be as descriptive as possible. This helps with readability.

Example 2

1. `// TODO: Compute Person Age`
2. `var birthYear = 1952; // BAT`
3. `const thisYear = new Date().getFullYear(); // 2026`
4. `const age = thisYear - birthYear; // 74`
5. `const canVote = age >= 18 ? "Sure" : "Nope";`
6. `console.log(thisYear, age, canVote);`

Comments

- ❑ Comments are primarily used to **document** code snippets.
- ❑ Comments can also be used to ignore code snippets during **debugging**.
- ❑ Inline comments begin with `// ...`
- ❑ Multi-line comments are wrapped within `/* ... */`

Data Types

1.2

Array (*List)

- ❑ An Array is an ordered *list* of values known as **elements**.
- ❑ Arrays can store elements of different data types (numbers, strings, booleans, etc).
- ❑ Array elements are accessed based on their **index**.
- ❑ The first element in an Array is located at **index[0]**.
- ❑ The last element in an Array is located at **index[n-1]**, where n is the Array size.
- ❑ The size of an Array in JavaScript changes as elements are added or removed.
- ❑ A nested Array is called a **Matrix**, i.e. an Array of Arrays.

Example 1

```
1. //TODO: Process Jamb Scores
2. const jambScores = [56, 42, 55, 60]; // ENG, MTH, PHY, ECO
3. var size = jambScores.length; // 4
4. var english = jambScores[0]; // 56
5. var economics = jambScores[size - 1]; // 60
6. var scored_56 = jambScores.includes(56); // false
7. var scored_76 = jambScores.includes(76); // true
8. var removedLastValue = jambScores.pop(); // [56, 42, 55], 60
9. var newArraySize = jambScores.push(55); // [56, 42, 55, 55], 4
10. var removedFirstValue = jambScores.shift(); // [42, 55, 55], 56
11. var newArraySize = jambScores.unshift(76); // [76, 42, 55, 55], 4
12. var sortedScoresAsc = jambScores.toSorted((a, b) => a - b); // [42, 55, 55, 76];
13. var sortedScoresDesc = jambScores.toSorted((a, b) => b - a); // [76, 55, 55, 42];
14. var totalScore = jambScores.reduce((total, value) => total + value, 0); // 228
15. var minScore = Math.min(...jambScores); // 42
16. var maxScore = Math.max(...jambScores); // 76
17. var verifyDatatype = Array.isArray(jambScores); // true
18. console.log(jambScores, totalScore, sortedScoresAsc, sortedScoresDesc);
```

Object (*HashMap)

- ❑ An Object is a record of **key:value** pairs called **properties**.
- ❑ Objects can store properties of different data types (numbers, strings, booleans, etc).
- ❑ Object properties are accessed based on their **key**, either using the “dot-notation” **record.key** (preferred) or the “index-notation” **record[key]** (dynamic).
- ❑ **Object.keys(record)** returns Object keys as Array, **Object.values(record)** also returns Object values as Array & **Object.entries(record)** returns Object properties as Matrix.
- ❑ **JSON.stringify(record)** returns Object properties as a serialized string.
- ❑ A nested Object is called a **Tree**, i.e. an Object of Objects.
- ❑ An “Array of Objects” is called a **Collection**.

Example 2

```
1.  // TODO: Process Jamb Scores
2.  const jambScores = { ENG: 56, MAT: 42, PHY: 55, ECO: 60 };
3.  var size = Object.keys(jambScores).length; // 4
4.  var english = jambScores["ENG"]; // 56 (index-notation)
5.  var economics = jambScores.ECO; // 60 (dot-notation preferred)

6.  jambScores.PHY = 71 // {ENG: 56, MAT: 42, PHY: 71, ECO: 60}
7.  jambScores.CHE = 35 // {ENG: 56, MAT: 42, PHY: 71, ECO: 60, CHE: 35}
8.  delete jambScores.CHE; // {ENG: 56, MAT: 42, PHY: 71, ECO: 60}
9.  var wroteChemistry = Object.keys(jambScores).includes("CHE"); // god forbid!
10. var scored_56 = Object.values(jambScores).includes(56); // true
11. var totalScore = Object.values(jambScores).reduce((total, value) => total + value, 0); // 229
12. var minScore = Math.min(...Object.values(jambScores)); // 42
13. var maxScore = Math.max(...Object.values(jambScores)); // 71
14. var verifyDatatype = typeof jambScores === 'object'; // true
15. console.log(jambScores, totalScore, wroteChemistry, verifyDatatype);
```

Date

- ❑ Date is an in-built JavaScript **module** for processing date and time.
- ❑ A Date instance can be created with **new Date()**, which returns the current browser/server timestamp.
And also with **new Date("YYYY-MM-DD")** OR **new Date("YYYY-MM-DD HH:MM:SS")**
- ❑ **new Date().toString()** returns **"Fri Jul 03 2026 04:00:12 GMT+0100 (West Africa Time)"**.
- ❑ **new Date().toISOString()** returns **"1970-01-01T00:00:00.000Z"** - The standard format for storing timestamps.
- ❑ **Date.now()** returns the total number of milliseconds since **Unix Epoch Time** (the beginning of time computations).
- ❑ A nested Date is called a **Calendar**, i.e. a Date of Dates? **Na lie o!** 😊

Example 3

```
1. // TODO: Format datetime as Sun, 8 Jul 2026 | 10:00 AM
2. const dateObj = new Date("2020-10-20 15:00:00"); // #EndSARS Protest
3. const dayOfWeekIndex = dateObj.getDay(); // 0-6
4. const dayOfMonth = dateObj.getDate(); // 1-31
5. const monthIndex = dateObj.getMonth(); // 0-11
6. const year = dateObj.getFullYear(); // 2026
7. const hourIndex = dateObj.getHours(); // 0-23
8. const minuteIndex = dateObj.getMinutes(); // 0-59
9.
   const days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
10.  const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
11.  const meridiem = hourIndex < 12 ? "AM" : "PM";
12.  const hours_f = hourIndex % 12 || 12; // 1-12
13.  const minutes_f = minuteIndex.toString().padStart(2, "0"); // 00-59
14.  const date_f = `${days[dayOfWeekIndex]}, ${dayOfMonth} ${months[monthIndex]} ${year}`;
15.  const time_f = `${hours_f}:${minutes_f} ${meridiem}`;
16.  const output = `${date_f} | ${time_f}`;
17.  console.log(dateObj.toString(), output);
```


1.3

Operators

Arithmetic

Oper	Name / Link	Example
+	Addition	$x = y + 2$
-	Subtraction	$x = y - 2$
*	Multiplication	$x = y * 2$
/	Division	$x = y / 2$
**	Exponentiation	$x = y ** 2$
%	Remainder	$x = y \% 2$
++	Increment	$x = y++$
--	Decrement	$x = y--$

Assignment

Oper	Name / Link	Example	Same As
=	Simple	x = y	x = y
+=	Add	x += y	x = x + y
-=	Subtract	x -= y	x = x - y
*=	Multiply	x *= y	x = x * y
/=	Divide	x /= y	x = x / y
%=	Remainder	x %= y	x = x % y

Comparison

Oper	Name / Link	Comparing	Returns
==	Equal to	x == 8	false
==	Equal to	x == 5	true
===	Strict Equal	x === "5"	false
===	Strict Equal	x === 5	true
!=	Not Equal	x != 8	true
!=	Strict not Equal	x != "5"	true
!=	Strict not Equal	x != 5	false
>	Greater than	x > 8	false
<	Less than	x < 8	true
>=	Greater or Equal	x >= 8	false
<=	Less or Equal	x <= 8	true

Logical

Oper	Name / Link	Example
&&	<u>AND</u>	(x < 10 && y > 1)
	<u>OR</u>	(x === 5 y === 5)
!	<u>NOT</u>	!(x === y)
??	<u>Nullish Coalescing</u>	x ?? y



Questions

2

BOM BOM & DOM

Browser/Document Object Model

3

Events Triggers & Handlers

4

**Vibe Coding
w/Claude AI**

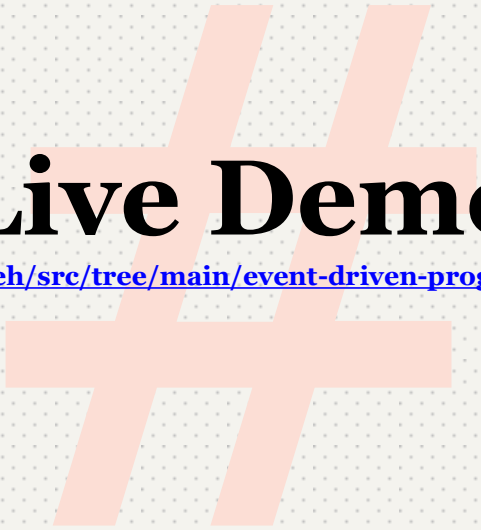
> _

[attach-screenshot] generate static html5 semantic tags and css3 in single file

Exercise 1

- ❑ **Northwind AI** is an AI Automation and Data Analytics service, provided by HWP Labs.
- ❑ **Northwind AI Podcast** is a platform that promotes tech startups, and application areas of AI within and outside the tech industry.
- ❑ Northwind AI Podcast holds LIVE on X Spaces, on Fridays & Sundays at 8pm (WAT).
- ❑ **Tech Stack:** TypeScript (Next.js), Python (FastAPI), PostgreSQL (Supabase) & Vercel
- ❑ **TODO:** “Vibe code” Northwind AI Podcast homepage using Claude AI, and implement load, input & click events with JavaScript.





Live Demo

<https://github.com/2gbeh/src/tree/main/event-driven-programming/northwind-ai>

Exercise 2

- ❑ **OPay** is a FinTech **super-app** founded in 2018, and acquired by Opera Software.
- ❑ OPay operates across Nigeria, Egypt, Pakistan and other markets offering Mobile Money (payments, transfers, loans, utility bills), ATM Cards, POS Terminals, Ride-hailing, and Food Delivery services.
- ❑ OPay reported serving 45m+ Users, and 1m+ Merchants & Businesses in Nigeria.
- ❑ **Tech Stack:** React Native, Kotlin, Swift, Node.js, PostgreSQL, Redis, Kafka & AWS
- ❑ **TODO:** “Vibe code” OPay App transfer screen using Claude AI, and implement *input* & *click* events with JavaScript.





Live Demo

<https://github.com/2gbeh/src/tree/main/event-driven-programming/opay>

Exercise 3

- ❑ **ExpoCBT** is an open-source CBT **WebApp** developed by E.O. Tugbeh & H.M. Oshomoh.
- ❑ ExpoCBT will be “**built in public**” on **TikTok LIVE**, alongside students of the Dept. of Software Engineering, Obafemi Awolowo University (OAU).
- ❑ Active participants will be added on GitHub as Contributors and assigned Issues.
- ❑ **Tech Stack:** TypeScript (Next.js), PostgreSQL (Supabase) & Vercel
- ❑ **TODO:** “Vibe code” ExpoCBT using Claude AI, implement load, input & click events with JavaScript, and deploy on a Live production env using Vercel.



Live Demo

<https://github.com/hwp-labs/expo-cbt>

NOTE: Create a GitHub account and **[Star]** the repository to be added as a Contributor!



Questions

Resources

1. Uvietobore, T. (2026). [WATCH] SEN106/SEN216 - Introduction to Web Technologies: <https://youtube.com/watch?v=cGoyAmGwDog>,
2. Tugbeh, E.O. (2026). [PDF] Event-Driven Programming: <https://github.com/2gbeh/src/blob/main/event-driven-programming/slide.pdf>
3. Tugbeh, E.O. (2026). [ZIP] Source Code Repository: <https://github.com/2gbeh/src/blob/main/event-driven-programming/lab.zip>
4. Tugbeh, E.O. (2026). Code Snippets Markdown File: <https://github.com/2gbeh/src/blob/main/event-driven-programming>
5. OneCompiler JavaScript Online Interpreter - <https://onecompiler.com/javascript>
6. Tugbeh, E.O., Oshomoh, H.M., & Aiyeduyoni S. (2026). Northwind AI Podcast E19 - Statement of Work: Government Subcontractors: <https://x.com/i/spaces/1wGWjjElmNNKQ>
7. Join Northwind AI Community: <https://chat.whatsapp.com/HajjJPJ2HkEorTTITv2FGm>
8. Support Northwind AI: <https://support.northwindai.org>
9. W3Schools – JavaScript Tutorial: <https://w3schools.com/js/default.asp>
10. W3Schools – Introduction to Programming: <https://w3schools.com/programming/index.php>

Let's Connect

Email	etugbeh@outlook.com
Telephone	(+234) 81 6996 0927
LinkedIn	/in/2gbeh
GitHub	/2gbeh
Twitter/X	/@2gbeh
TikTok	/@2gbeh
Website	https://northwindai.org
Podcast	https://podcast.northwindai.org
Community	https://chat.whatsapp.com/HajjJPJ2HkEorTTITv2FGm

Thank You!

